Atlas-Ingest: Technical Architecture & Production Design

This document details the production architecture for the Atlas-Ingest data ingestion pipeline, designed for resilience, horizontal scalability, and high-fidelity extraction.

> **Note:** Sections 1, 3, and 4 describe the **production roadmap** for scaling Atlas-Ingest to 500,000+ records. The current implementation covers the core pipeline (crawling, LLM extraction, entity resolution, and export). The scaling strategies below are designed and documented for future implementation.

1. Scale Strategy (500,000+ Records) — *Planned*

To scale this architecture to "lakhs" of records without manual intervention or code changes, the roadmap transitions from a monolithic local execution model to a distributed, event-driven microservices architecture. - **Message Broker (Kafka/RabbitMQ):** URL seeds and API tasks are pushed to distributed queues (e.g., `url_ingestion_queue`, `llm_processing_queue`). - **Horizontal Scaling:** We deploy stateless crawler workers on Kubernetes (K8s). As the queue depth increases, Kubernetes Horizontal Pod Autoscaler (HPA) dynamically provisions more crawler pods. - **Proxy Rotation & IP Management:** For heavy directories (ProductHunt, YC), workers route requests through a residential proxy network (e.g., BrightData or Oxylabs) to distribute IP fingerprints and avoid Datadome/Cloudflare blanket bans.

2. Handling 413s & 429s (LLM Resilience)

When processing thousands of concurrent extractions via LLMs, context window overflows (413 Payload Too Large) and Rate Limits (429 Too Many Requests) are inevitable. - **Handling 413s (Intelligent Chunking):** We utilize `tiktoken` to count tokens *before* dispatching payloads to the LLM. Using our `ContentChunker`, raw HTML is stripped of visual bloat (CSS/JS) and sliced into overlapping semantic windows (e.g., 4000 tokens) that guarantee the prompt never exceeds the context limit. - **Handling 429s (Exponential Backoff + Fallback):** We use a Multi-Tier Fallback Chain managed by `LiteLLM` and `Tenacity`. If our primary cheap/fast model (e.g., `gemini-1.5-flash`) hits a 429, `Tenacity` triggers an exponential backoff with jitter. If the limit remains exhausted, the orchestrator

gracefully cascades to the next tier (e.g., `groq/llama-3.1`, then `deepseek`), ensuring the pipeline never halts.

3. Freshness Tracking (Distributed Consistency)

To ensure we never process the same article or job twice across distributed crawler nodes, we must implement a centralized, fast-access tracking system. - **Redis Bloom Filters:** We utilize a Redis Bloom Filter to track URLs and content hashes. Bloom filters are highly memory-efficient for checking the existence of millions of URLs in O(1) time. Before a worker scrapes a link, it checks the filter. - **Content Hashing:** Because URLs sometimes change for the same article, we generate a SHA-256 hash of the normalized article title/company name. This hash is stored in Redis with a 48-hour TTL. Any incoming item matching a known hash is instantly discarded as stale.

4. Storage Strategy

Building an Intelligence Graph requires capturing both the raw entities and the complex relationships between them (e.g., Founder -> Startup -> Product). - **Primary Database (PostgreSQL / JSONB):** PostgreSQL serves as our primary OLTP database. It offers ACID compliance for resilient writes. We utilize its `JSONB` column type to store the polymorphic payload extracted from the LLM, giving us the flexibility of NoSQL with the rigorous schema enforcement of SQL. - **Graph Storage (Neo4j):** To query complex, multi-hop relationships (e.g., "Find all Research Papers authored by Founders of YC Startups"), we sync canonicalized entities to Neo4j. It natively supports graph traversals which are computationally prohibitive in relational databases. - **Vector Storage (Pinecone / pgvector):** For semantic search (e.g., "Find startups similar to OpenAI"), we embed the LLM-extracted summaries using a model like `text-embedding-3-small` and store the vectors in `pgvector` alongside the relational data to maintain operational simplicity.